MODULE 08 · DOMAIN 6 - PROMPT AND CONTEXT ENGINEERING

Context Engineering

Managing what Claude sees across turns, tools and steps: drift, bloat, pruning, compaction, isolation and retrieval.

DOMAIN 6 of 8 EXAM WEIGHT 11.0% MODULE 8 of 36

admin@sthithapragna.com

Prompt engineering writes one good request. Context engineering decides **what Claude sees** at every step.

Prompt engineering or context engineering?

PROMPT ENGINEERING

One request

- Shapes a single prompt
- Wording, examples, format
- Judged on one output
- Module 5

CONTEXT ENGINEERING

Every step

- Manages what flows across turns and tools
- Keeps relevant state visible
- Prunes, summarises, isolates
- Long chats and multi-step agents

THE TIEBREAKER

Same model, same prompt, worse answers by step forty? That is a context problem, not a prompt problem.

The window is a budget

- **Anthropic's goal:** "the smallest possible set of high-signal tokens" that gets the job done.
- **Context rot:** as token count grows, accuracy and recall degrade.
- **Every turn resends everything:** history, tool results, documents, earlier thinking.
- **A million tokens is capacity, not a plan.** Long tasks fill it faster than you think.

> THE TAKEAWAY
>
> Treat context like memory in a tight loop: allocate it on purpose, and free what you no longer need.

How long work goes wrong

PROBLEM	WHAT YOU SEE	USUAL CAUSE
Drift	Format slips after many turns	Rules buried under history
Drift	Early facts and decisions forgotten	Key state only in turn one
Bloat	Cost and latency climb each turn	Spent tool output kept forever
Bloat	Answers get vaguer as context grows	Rot from low-signal tokens

ON THE EXAM

Drift and bloat are context-management problems. A bigger model or a louder prompt rarely fixes either.

The context engineering toolkit

TECHNIQUE	USE IT WHEN
Prune tool output	Old tool results are no longer needed by later steps
Context editing	You want the API to clear old tool results or thinking by rule
Compaction	A long conversation must continue, and its gist matters
Pinned state	A few facts must stay visible every turn
Isolation	A step should see only what it needs
Just-in-time retrieval	Load data when a step needs it, not all up front

ON THE EXAM

Each technique answers one symptom. Match them, and combine them in long-running agents.

Context editing: clearing old tool results

```python
response = client.beta.messages.create(
    model="claude-opus-5-5", max_tokens=4096,
    betas=["context-management-2025-06-27"],
    context_management={"edits": [{
        "type": "clear_tool_uses_20250919",
        "trigger": {"type": "input_tokens", "value": 30000},
        "keep": {"type": "tool_uses", "value": 3},
        "clear_at_least": {"type": "input_tokens", "value": 5000},
        "exclude_tools": ["get_tenancy"],
    }]},
    tools=TOOLS, messages=history,
)
```

6-7 Once input passes the trigger, clear old results, keeping the last three.

9 Results from this tool are never cleared.

CURRENCY CHECK

Beta, and applied server-side: your code keeps the full history. Clearing invalidates the cache, so clear in sizeable chunks.

Compaction

- **Replaces older turns with a summary** Claude writes on the server, so the conversation can continue.
- **Why:** "response quality degrades as a conversation grows".
- **Two forms, both beta:** on demand, where you choose when, and at a token threshold, where the API does.
- **Custom summary prompt** when the default drops something later turns need.
- **Elsewhere:** the Agent SDK compacts automatically; Claude Code has a compact command.

> ON THE EXAM
>
> Anthropic: for most long-running conversations, server-side compaction is the primary context strategy.

Prune, or compact?

PRUNE OR CLEAR

Remove by rule

- Old tool results or thinking
- Precise and predictable
- No summary is written
- Best for agents with bulky tool output

COMPACT

Summarise the past

- Older turns become a summary
- Keeps the gist and decisions
- Summary quality matters
- Best for long conversations

THE TIEBREAKER

Bulky output you no longer need: prune. A long conversation whose gist still matters: compact.

What must survive

- **Standing rules in the system prompt,** never only in the first user turn.
- **Pinned state:** a short block of key facts re-sent every turn - who, what, which property.
- **Decisions and commitments** written into any summary, by your summarisation prompt.
- **For agents:** notes or a memory tool hold progress outside the window.
- **Claude Code and the Agent SDK** re-inject CLAUDE.md on every request, so it survives compaction.

THE TAKEAWAY

Decide what must never be lost, and put it where trimming and summarising cannot reach it.

Isolation: each step sees only what it needs

- **Separate calls or subagents,** each with its own small context.
- **Subagents return a condensed summary,** often 1,000 to 2,000 tokens, not their whole working.
- **Keeps irrelevant or sensitive context** out of later reasoning.
- **Costs more tokens overall,** so isolate where focus or confidentiality is worth it.

> ON THE EXAM
>
> Isolation is the guide's answer when a step must reason without the full prior history.

Just-in-time retrieval

- **Load on demand:** give Claude tools to fetch what a step needs, when it needs it.
- **Instead of** pasting every document, record and log into the first request.
- **The hybrid:** a small, stable core up front, details fetched as needed.
- **Trade-off:** extra steps and a risk of missing something, against a leaner, sharper context.

> THE TAKEAWAY
>
> Claude Code works this way: CLAUDE.md up front, files found with search tools when needed.

Whole document, or chunks?

SEND IT WHOLE

Reason across sections

- Clauses that refer to each other
- Cross-section conclusions
- Risk: detail drifts in later sections
- Fits the window? Then it is possible

SEND CHUNKS

Precision per piece

- Tighter citations per chunk
- Smaller, cheaper calls
- Risk: cross-references lost
- Needs a merge step

THE TIEBREAKER

Decide which property the use case needs most, choose accordingly, then validate on representative documents.

Copperlet's forgetful assistant, fixed

BEFORE

Everything, forever

- Rules sent in the first user turn
- Tenant and property named once
- Full history every turn
- Format drifting by turn twelve

AFTER

Curated every turn

- Rules in the system prompt
- Pinned tenant context block each turn
- Compaction on demand after 20 turns
- Summary keeps open repairs and promises

SETTLED

The assistant now knows who it is talking to at turn fifty.

> An investigation agent calls a log-search tool many times per task. Each call returns several thousand tokens of log lines. Later steps only need the last few results and a short list of findings.
>
> Cost and latency rise steeply as each task goes on.

What should the developer do?

A. Prune spent tool results, for example with context editing that keeps only the last few tool uses

B. Cache the accumulated tool results so that the old content is cheaper to resend

C. Move the agent to a smaller model and accept any loss in quality

D. Raise max tokens so that the agent has more room to work

◆ PAUSE AND DECIDE

What should the developer do?

A Prune spent tool results, for example with context editing that keeps only the last few tool uses

Removes tokens later steps never use. Cheaper, faster, sharper.

B Cache the accumulated tool results so that the old content is cheaper to resend

Cheaper bloat is still bloat, and it still rots the context.

C Move the agent to a smaller model and accept any loss in quality

Treats the symptom and accepts a quality loss nobody asked for.

D Raise max tokens so that the agent has more room to work

max_tokens caps output. The bloat is input.

THE SIGNAL

"Later steps only need the last few results" is the signal for pruning.

> A team's prompts score well on single-turn tests. But in a 40-step research agent, quality falls in later steps: the agent repeats searches and loses findings from early steps.
>
> The lead proposes another round of prompt rewriting.

What should the team focus on instead?

A Rewriting the system prompt again, with stronger emphasis on not repeating searches

B Moving to a model with a larger context window so nothing is ever lost

C Context engineering: keep findings in notes, prune spent results, and control what each step sees

D Lowering the number of steps by banning searches after step twenty

◆ PAUSE AND DECIDE

What should the team focus on instead?

A Rewriting the system prompt again, with stronger emphasis on not repeating searches

The prompt already works on single turns. The problem is across steps.

B Moving to a model with a larger context window so nothing is ever lost

Capacity is not the issue. More context can mean more rot.

C Context engineering: keep findings in notes, prune spent results, and control what each step sees

Manages state across steps, which is where quality is lost.

D Lowering the number of steps by banning searches after step twenty

An arbitrary limit that may stop valid work.

THE SIGNAL

Good on single turns, bad by step forty: a context problem, not a prompt problem.

> A booking assistant returns well-formed JSON for the first few turns of a conversation. After many turns, responses gradually lose structure. The format rules appear only in the first user message.
>
> The team wants format to hold for the whole conversation.

Which two changes address the drift? Choose two.

A. Move the format rules into the system prompt rather than only the first user message

B. Lower the temperature as the conversation gets longer

C. Restart the application after every turn

D. Switch to a smaller model, which follows formats more strictly

E. Validate each response's format, and trim or compact older turns as the conversation grows

◆ PAUSE AND DECIDE

Which two changes address the drift? Choose two.

A Move the format rules into the system prompt rather than only the first user message

Standing rules stay prominent instead of sinking under history.

B Lower the temperature as the conversation gets longer

Wrong layer, and not accepted by current models.

C Restart the application after every turn

Destroys the conversation it was meant to protect.

D Switch to a smaller model, which follows formats more strictly

No such rule. The cause is context, not model size.

E Validate each response's format, and trim or compact older turns as the conversation grows

Catches drift when it happens and removes what feeds it.

THE SIGNAL

Name the failure mode - format drift - then fix where the rules live and how context grows.

> An insurer's claims workflow extracts facts, then runs an eligibility check, then drafts a decision. The eligibility step must judge only the structured facts, not the customer's emotional free-text appeal, which has biased it before.
>
> All three steps currently share one long conversation.

What should the developer do?

A Keep one conversation, and tell the eligibility step to ignore the appeal

B Give every step the full history, so nothing is missing from any decision

C Use one global system prompt that covers all three steps equally

D Run eligibility as a separate call that receives only the structured facts it needs

◆ PAUSE AND DECIDE

What should the developer do?

A Keep one conversation, and tell the eligibility step to ignore the appeal

Asks Claude to unsee what is in front of it. That already failed.

B Give every step the full history, so nothing is missing from any decision

Exactly the leak that caused the bias.

C Use one global system prompt that covers all three steps equally

Does nothing about what the eligibility step can see.

D Run eligibility as a separate call that receives only the structured facts it needs

Context isolation: the appeal is simply not there to bias it.

THE SIGNAL

"Must judge only the structured facts" means isolate the step, not instruct it.

> A legal-tech team summarises 70-page leases with citations. Sending the whole lease gives better cross-clause reasoning, but citations drift in later sections. Chunking gives precise citations, but misses clauses that refer to each other.
>
> The team has not agreed which property matters more.

How should the team decide?

A Decide which property the use case needs most, choose the approach that serves it, and validate on representative leases

B Always chunk, because citations are the most important feature of any legal summary

C Always send the whole lease, because it fits inside the context window

D Choose whichever approach is cheaper per lease, and revisit later

◆ PAUSE AND DECIDE

How should the team decide?

A Decide which property the use case needs most, choose the approach that serves it, and validate on representative leases

Requirement first, then choose, then validate. Rule three.

B Always chunk, because citations are the most important feature of any legal summary

Assumes the requirement instead of deciding it.

C Always send the whole lease, because it fits inside the context window

Fitting makes it possible, not right.

D Choose whichever approach is cheaper per lease, and revisit later

Optimises a property nobody named as the goal.

THE SIGNAL

"Has not agreed which property matters" is the signal: settle the requirement first.

> A property-management agent compacts its conversation once it grows long. After compaction, it sometimes forgets a repair appointment it had already agreed with the tenant, and offers a new one.
>
> Everything else about its behaviour is fine.

What is the best fix?

A. Turn compaction off and let the context grow until the window is full

B. Lower the compaction threshold so that it compacts more often

C. Move the agent to the largest model so the summary is more thorough

D. Write a custom summarisation prompt that always keeps commitments, open actions and key identifiers

◆ PAUSE AND DECIDE

What is the best fix?

A Turn compaction off and let the context grow until the window is full

Trades one failure for rot and an eventual overflow.

B Lower the compaction threshold so that it compacts more often

More summaries, each still dropping the appointment.

C Move the agent to the largest model so the summary is more thorough

Hopes for thoroughness instead of specifying what to keep.

D Write a custom summarisation prompt that always keeps commitments, open actions and key identifiers

Anthropic's documented fix when the default summary drops what later turns need.

THE SIGNAL

The summary dropped something specific. Tell the summariser what must survive.

Module 8 in five lines

- **Context engineering manages what Claude sees** across turns, tools and steps; prompt engineering shapes one request.
- **Drift and bloat** are context problems. The window is a budget, not a bin.
- **Prune bulky output** (or use context editing); **compact long conversations**, with a summary prompt that keeps what matters.
- **Pin key state, keep rules in the system prompt,** and isolate steps that must not see everything.
- **Whole or chunked:** decide the property you need, then validate.

COMING UP

Next: from business goal to requirements - the start of Part 3, building the application.